Estruturas de Dados com C++

Jonathan Tenório de Lima

24 de março de 2026

Sumário

1 Ordenação **2**
1.1 Insertion sort 2
1.2 Bubble sort 3

1 Ordenação

1.1 Insertion sort

Esse algoritmo de ordenação funciona semelhante a organizar as cartas do baralho na mão, durante uma partida. Inicialmente, as cartas podem estar desordenadas, então, você seleciona a segunda e compara com aquelas à esquerda, movendo a carta até a posição em esta seja a menor da sequência. A cada intervalo de interação, a carta de comparação pode ser chamada de ***carta*** e o vetor contendo as cartas será denominado $\boldsymbol{v}$. A figura 1 exemplifica o funcionamento do método de ordenação. A cada tempo, a *carta* selecionada é deslocada até a posição correspondente ordenada, o que "empurra"as outras cartas para uma posição mais adiante.

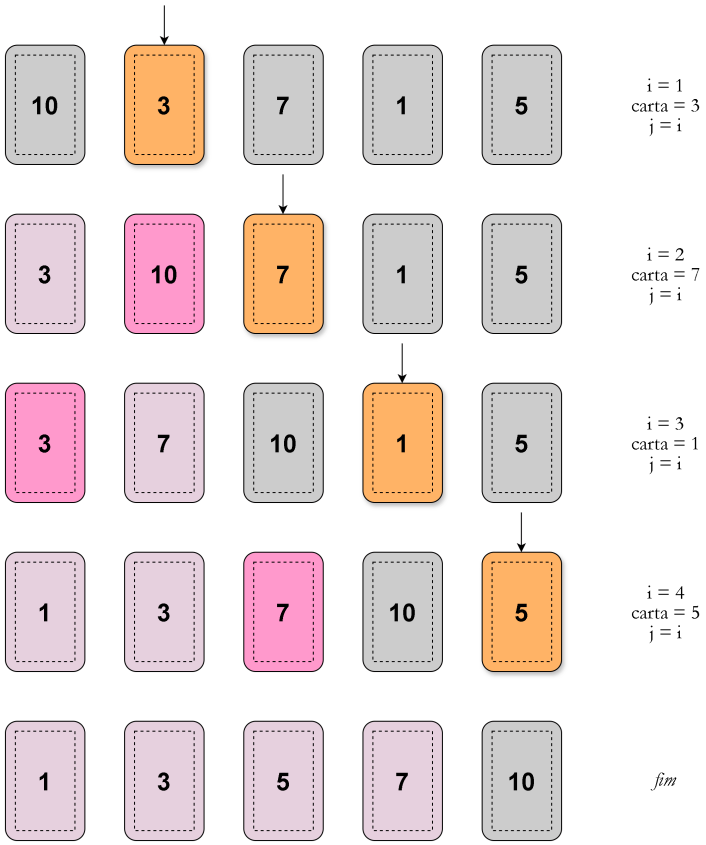

Figura 1: Exemplo do funcionamento do Insertion Sort

O pseudocódigo deste algoritmo pode ser dado como:

Algorithm 1 Insertion Sort

```
procedure INSERTIONSORT(v)
    n ← tamanho(v)
    for i ← 1 to n − 1 do
        carta ← v[i]
        j ← i
        while j > 0 and v[j − 1] > carta do
            v[j] ← v[j − 1]
            j ← j − 1
        end while
        v[j] ← carta
    end for
end procedure
```

Também é possível escrever o código correspondente em C++ como a seguir:

```cpp

void insertionSort( std::vector<int>& v ) {
    for ( int i = 1; i < n; ++i ){
        auto carta = v[i];
        auto j = i;
        while ( j > 0 and v[j-1] > carta ) {
            v[j] = v[j-1];
            j = j - 1;
        }
        v[j] = carta;
    }
}
```

1.2 Bubble sort

Algorithm 2 Bubble Sort

```
procedure BUBBLESORT(v)
    rlim ← v.size()                    ▷ Define o limite a direita (right limite, rlim
    for i ← 1 to n − 1 do
        lsp ← 0                        ▷ Define a última posição de troca, lsp
        for i ← 1 to rlim do
            if v[i − 1] > v[i] then
                aux = v[i − 1]
                v[i − 1] = v[i]
                v[i] = aux
                lsp = i                ▷ lsp recebe a última posição em que ocorreu troca
            end if
        end for
        rlim ← lsp                     ▷ Define o novo limite à direita como a última posição de troca
    end for
end procedure
```

```

void bubbleSort( std::vector<int>& v ) {

    int rlim = v.size();  // limite direito

    for ( std::size_t j = 0; j < v.size(); ++j ) {
            std::size_t lsp = 0;
            // lsp: last swap position
            for ( auto i = 1; i < rlim; ++i ) {
                if ( v[i-1] > v[i] ) {
                    auto aux = v[i-1];
                    v[i-1] = v[i];
                    v[i] = aux;
                    lsp = i;
                }
            }
            rlim = lsp;
        }
```